

Comparison of GenAI Programs

Prompt

Write a C program for an EECS 348 Software Engineering assignment. The program will prioritize emails for a busy company CEO using a priority queue implemented as a list-based MaxHeap. You must implement the MaxHeap and all heap operations from scratch. Do not use any pre-existing heap, priority queue, or similar data-structure modules.

The input is a test file containing email records followed by commands. Each email has this format:

```
EMAIL <sender category>,<subject line>,<date>
```

The sender category will be exactly one of: Boss, Subordinate, Peer, ImportantPerson, OtherPerson.

Priority order from highest to lowest is: Boss, Subordinate, Peer, ImportantPerson, OtherPerson.

If two or more emails have the same sender category, the newest email must have higher priority than the older email. Dates are in MM-DD-YYYY format. The subject line may contain spaces but will not contain commas.

The program must support these commands:

NEXT Display the next email the CEO should read, but do not remove it from the queue. The output format must be:

```
Next email:
Sender:  <sender>
Subject: <subject>
Date:   <date>
```

READ The CEO has read and dealt with the highest-priority email, so remove that email from the queue. If READ is called without a previous NEXT, simply remove the highest-priority email without displaying it.

COUNT Display the current number of unread emails in the queue in this format:

```
There are <count> emails to read.
```

The program must correctly handle: NEXT when there are no emails; READ when there are no emails; two NEXT commands in a row without READ (both NEXT commands should display the same email); two READ commands in a row without NEXT (each READ should remove the current highest-priority email); and any order and number of commands in the test file.

Use a list-based MaxHeap as the underlying priority queue. Do not hard-code the sample input or assume a fixed number or ordering of commands.

Gemini

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAX_LINE 512
6 #define INITIAL_CAPACITY 16
7
8 // Structure to store Email data
9 typedef struct {
10     char sender[32];
11     char subject[256];
12     char date[11]; // Format: MM-DD-YYYY
13     int year;
14     int month;
15     int day;
16     int priority_score;
17 } Email;
18
19 // Dynamic array-based MaxHeap
20 typedef struct {
21     Email *data;
22     int size;
23     int capacity;
24 } MaxHeap;
25
26 // Function Prototypes
```

```

27 MaxHeap* create_heap();
28 void destroy_heap(MaxHeap *heap);
29 void insert_email(MaxHeap *heap, Email email);
30 Email extract_max(MaxHeap *heap);
31 Email* peek_max(MaxHeap *heap);
32 void heapify_up(MaxHeap *heap, int index);
33 void heapify_down(MaxHeap *heap, int index);
34 int compare_emails(Email a, Email b);
35 int get_priority_score(const char *sender);
36 void parse_date(const char *date_str, int *year, int *month, int *day);
37
38 // Assign numeric scores for sender categories
39 int get_priority_score(const char *sender) {
40     if (strcmp(sender, "Boss") == 0) return 5;
41     if (strcmp(sender, "Subordinate") == 0) return 4;
42     if (strcmp(sender, "Peer") == 0) return 3;
43     if (strcmp(sender, "ImportantPerson") == 0) return 2;
44     if (strcmp(sender, "OtherPerson") == 0) return 1;
45     return 0;
46 }
47
48 // Parse MM-DD-YYYY into integer components for easy comparison
49 void parse_date(const char *date_str, int *year, int *month, int *day) {
50     sscanf(date_str, "%2d-%2d-%4d", month, day, year);
51 }
52
53 // Returns > 0 if email 'a' has higher priority than email 'b'
54 int compare_emails(Email a, Email b) {
55     if (a.priority_score != b.priority_score) {
56         return a.priority_score - b.priority_score;
57     }
58     // Tie-breaker: Newest date first
59     if (a.year != b.year) return a.year - b.year;
60     if (a.month != b.month) return a.month - b.month;
61     return a.day - b.day;
62 }
63
64 MaxHeap* create_heap() {
65     MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap));
66     heap->capacity = INITIAL_CAPACITY;
67     heap->size = 0;
68     heap->data = (Email*)malloc(heap->capacity * sizeof(Email));
69     return heap;
70 }
71
72 void destroy_heap(MaxHeap *heap) {
73     free(heap->data);
74     free(heap);
75 }
76
77 void heapify_up(MaxHeap *heap, int index) {
78     while (index > 0) {
79         int parent = (index - 1) / 2;
80         if (compare_emails(heap->data[index], heap->data[parent]) > 0) {
81             Email temp = heap->data[index];
82             heap->data[index] = heap->data[parent];
83             heap->data[parent] = temp;
84             index = parent;
85         } else {
86             break;
87         }
88     }
89 }
90
91 void heapify_down(MaxHeap *heap, int index) {
92     while (1) {
93         int left = 2 * index + 1;
94         int right = 2 * index + 2;
95         int largest = index;
96
97         if (left < heap->size && compare_emails(heap->data[left], heap->data[largest]) > 0) {
98             largest = left;
99         }
100         if (right < heap->size && compare_emails(heap->data[right], heap->data[largest]) > 0) {
101             largest = right;
102         }
103
104         if (largest != index) {
105             Email temp = heap->data[index];
106             heap->data[index] = heap->data[largest];
107             heap->data[largest] = temp;
108             index = largest;
109         } else {
110             break;
111         }
112     }
113 }
114
115 void insert_email(MaxHeap *heap, Email email) {
116     if (heap->size >= heap->capacity) {

```

```

117     heap->capacity *= 2;
118     heap->data = (Email*)realloc(heap->data, heap->capacity * sizeof(Email));
119 }
120 heap->data[heap->size] = email;
121 heapify_up(heap, heap->size);
122 heap->size++;
123 }
124
125 Email extract_max(MaxHeap *heap) {
126     Email max_item = heap->data[0];
127     heap->data[0] = heap->data[heap->size - 1];
128     heap->size--;
129     heapify_down(heap, 0);
130     return max_item;
131 }
132
133 Email* peek_max(MaxHeap *heap) {
134     if (heap->size == 0) return NULL;
135     return &heap->data[0];
136 }
137
138 int main(int argc, char *argv[]) {
139     if (argc < 2) {
140         printf("Usage: %s <input_filename>\n", argv[0]);
141         return 1;
142     }
143
144     FILE *file = fopen(argv[1], "r");
145     if (!file) {
146         perror("Error opening file");
147         return 1;
148     }
149
150     MaxHeap *heap = create_heap();
151     char line[MAX_LINE];
152
153     while (fgets(line, sizeof(line), file)) {
154         // Strip trailing newline character
155         line[strcspn(line, "\r\n")] = 0;
156         if (strlen(line) == 0) continue;
157
158         if (strncmp(line, "EMAIL ", 6) == 0) {
159             Email email;
160             char *token_str = line + 6; // Move past "EMAIL "
161
162             // Extract sender
163             char *commal = strchr(token_str, ',');
164             if (!commal) continue;
165             *commal = '\0';
166             strcpy(email.sender, token_str);
167
168             // Extract subject
169             char *token_subject = commal + 1;
170             char *comma2 = strchr(token_subject, ',');
171             if (!comma2) continue;
172             *comma2 = '\0';
173             strcpy(email.subject, token_subject);
174
175             // Extract date
176             char *token_date = comma2 + 1;
177             strcpy(email.date, token_date);
178
179             // Assign numerical metadata
180             email.priority_score = get_priority_score(email.sender);
181             parse_date(email.date, &email.year, &email.month, &email.day);
182
183             insert_email(heap, email);
184         }
185         else if (strcmp(line, "NEXT") == 0) {
186             Email *next_email = peek_max(heap);
187             if (next_email == NULL) {
188                 printf("No emails to read.\n");
189             }
190             else {
191                 printf("Next email:\n");
192                 printf("Sender: %s\n", next_email->sender);
193                 printf("Subject: %s\n", next_email->subject);
194                 printf("Date: %s\n", next_email->date);
195             }
196         }
197         else if (strcmp(line, "READ") == 0) {
198             if (heap->size > 0) {
199                 extract_max(heap);
200             }
201         }
202         else if (strcmp(line, "COUNT") == 0) {
203             printf("There are %d emails to read.\n", heap->size);
204         }
205     }
206     fclose(file);

```

```

207     destroy_heap(heap);
208     return 0;
209 }

```

Claude

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <ctype.h>
5
6  #define MAX_LINE    1024
7  #define MAX_SUBJECT 512
8  #define MAX_SENDER  64
9  #define MAX_DATE     16
10 #define NUM_CATEGORIES 5
11
12 static const char *CATEGORY_NAMES[NUM_CATEGORIES] = {
13     "Boss",
14     "Subordinate",
15     "Peer",
16     "ImportantPerson",
17     "OtherPerson"
18 };
19
20 typedef struct {
21     int month;
22     int day;
23     int year;
24 } Date;
25
26 typedef struct {
27     char sender[MAX_SENDER];
28     char subject[MAX_SUBJECT];
29     char date_text[MAX_DATE];
30     Date date;
31     int rank;
32     long sequence;
33 } Email;
34
35 typedef struct {
36     Email *items;
37     int size;
38     int capacity;
39 } MaxHeap;
40
41 static int email_compare(const Email *a, const Email *b)
42 {
43     if (a->rank != b->rank)
44         return (a->rank < b->rank) ? 1 : -1;
45
46     if (a->date.year != b->date.year)
47         return (a->date.year > b->date.year) ? 1 : -1;
48     if (a->date.month != b->date.month)
49         return (a->date.month > b->date.month) ? 1 : -1;
50     if (a->date.day != b->date.day)
51         return (a->date.day > b->date.day) ? 1 : -1;
52
53     if (a->sequence != b->sequence)
54         return (a->sequence > b->sequence) ? 1 : -1;
55
56     return 0;
57 }
58
59 static void heap_init(MaxHeap *h)
60 {
61     h->items = NULL;
62     h->size = 0;
63     h->capacity = 0;
64 }
65
66 static void heap_free(MaxHeap *h)
67 {
68     free(h->items);
69     h->items = NULL;
70     h->size = 0;
71     h->capacity = 0;
72 }
73
74 static int heap_is_empty(const MaxHeap *h)
75 {
76     return h->size == 0;
77 }
78
79 static int heap_count(const MaxHeap *h)
80 {
81     return h->size;
82 }

```

```

83
84 static int heap_reserve(MaxHeap *h, int needed)
85 {
86     int new_capacity;
87     Email *grown;
88
89     if (needed <= h->capacity)
90         return 1;
91
92     new_capacity = (h->capacity == 0) ? 8 : h->capacity * 2;
93     while (new_capacity < needed)
94         new_capacity *= 2;
95
96     grown = (Email *)realloc(h->items, (size_t)new_capacity * sizeof(Email));
97     if (grown == NULL)
98         return 0;
99
100     h->items = grown;
101     h->capacity = new_capacity;
102     return 1;
103 }
104
105 static void heap_swap(MaxHeap *h, int i, int j)
106 {
107     Email temp = h->items[i];
108     h->items[i] = h->items[j];
109     h->items[j] = temp;
110 }
111
112 static void sift_up(MaxHeap *h, int index)
113 {
114     while (index > 0) {
115         int parent = (index - 1) / 2;
116         if (email_compare(&h->items[index], &h->items[parent]) > 0) {
117             heap_swap(h, index, parent);
118             index = parent;
119         } else {
120             break;
121         }
122     }
123 }
124
125 static void sift_down(MaxHeap *h, int index)
126 {
127     for (;;) {
128         int left = 2 * index + 1;
129         int right = 2 * index + 2;
130         int largest = index;
131
132         if (left < h->size &&
133             email_compare(&h->items[left], &h->items[largest]) > 0)
134             largest = left;
135
136         if (right < h->size &&
137             email_compare(&h->items[right], &h->items[largest]) > 0)
138             largest = right;
139
140         if (largest == index)
141             break;
142
143         heap_swap(h, index, largest);
144         index = largest;
145     }
146 }
147
148 static int heap_insert(MaxHeap *h, const Email *e)
149 {
150     if (!heap_reserve(h, h->size + 1))
151         return 0;
152
153     h->items[h->size] = *e;
154     h->size++;
155     sift_up(h, h->size - 1);
156     return 1;
157 }
158
159 static int heap_peek(const MaxHeap *h, Email *out)
160 {
161     if (heap_is_empty(h))
162         return 0;
163
164     *out = h->items[0];
165     return 1;
166 }
167
168 static int heap_remove_max(MaxHeap *h, Email *out)
169 {
170     if (heap_is_empty(h))
171         return 0;
172

```

```

173     if (out != NULL)
174         *out = h->items[0];
175
176     h->items[0] = h->items[h->size - 1];
177     h->size--;
178
179     if (h->size > 0)
180         sift_down(h, 0);
181
182     return 1;
183 }
184
185 static char *trim(char *s)
186 {
187     char *end;
188
189     while (*s != '\0' && isspace((unsigned char)*s))
190         s++;
191
192     if (*s == '\0')
193         return s;
194
195     end = s + strlen(s) - 1;
196     while (end > s && isspace((unsigned char)*end))
197         end--;
198     end[1] = '\0';
199
200     return s;
201 }
202
203 static void copy_bounded(char *dest, size_t dest_size, const char *src)
204 {
205     size_t i;
206     for (i = 0; i + 1 < dest_size && src[i] != '\0'; i++)
207         dest[i] = src[i];
208     dest[i] = '\0';
209 }
210
211 static int category_rank(const char *name)
212 {
213     int i;
214     for (i = 0; i < NUM_CATEGORIES; i++) {
215         if (strcmp(name, CATEGORY_NAMES[i]) == 0)
216             return i;
217     }
218     return -1;
219 }
220
221 static int parse_date(const char *text, Date *out)
222 {
223     int month = 0, day = 0, year = 0;
224     char extra;
225
226     if (sscanf(text, "%d-%d-%d%c", &month, &day, &year, &extra) != 3)
227         return 0;
228
229     if (month < 1 || month > 12 || day < 1 || day > 31 || year < 0)
230         return 0;
231
232     out->month = month;
233     out->day = day;
234     out->year = year;
235     return 1;
236 }
237
238 static int parse_email(char *body, long sequence, Email *out)
239 {
240     char *first_comma, *second_comma;
241     char *sender, *subject, *date_text;
242     int rank;
243
244     first_comma = strchr(body, ',');
245     if (first_comma == NULL)
246         return 0;
247     *first_comma = '\0';
248
249     second_comma = strchr(first_comma + 1, ',');
250     if (second_comma == NULL)
251         return 0;
252     *second_comma = '\0';
253
254     sender = trim(body);
255     subject = trim(first_comma + 1);
256     date_text = trim(second_comma + 1);
257
258     rank = category_rank(sender);
259     if (rank < 0)
260         return 0;
261
262     if (!parse_date(date_text, &out->date))

```

```

263     return 0;
264
265     copy_bounded(out->sender,    sizeof(out->sender),    sender);
266     copy_bounded(out->subject,   sizeof(out->subject),   subject);
267     copy_bounded(out->date_text, sizeof(out->date_text), date_text);
268     out->rank    = rank;
269     out->sequence = sequence;
270
271     return 1;
272 }
273
274 static void do_next(const MaxHeap *h)
275 {
276     Email top;
277
278     if (!heap_peek(h, &top)) {
279         printf("No emails to read.\n");
280         return;
281     }
282
283     printf("Next email:\n");
284     printf("Sender: %s\n", top.sender);
285     printf("Subject: %s\n", top.subject);
286     printf("Date: %s\n", top.date_text);
287 }
288
289 static void do_read(MaxHeap *h)
290 {
291     heap_remove_max(h, NULL);
292 }
293
294 static void do_count(const MaxHeap *h)
295 {
296     printf("There are %d emails to read.\n", heap_count(h));
297 }
298
299 static void process_stream(FILE *in, MaxHeap *heap)
300 {
301     char line[MAX_LINE];
302     long line_number = 0;
303     long arrival     = 0;
304
305     while (fgets(line, sizeof(line), in) != NULL) {
306         char *text;
307         line_number++;
308
309         text = trim(line);
310         if (*text == '\0')
311             continue;
312
313         if (strncmp(text, "EMAIL", 5) == 0 &&
314             (text[5] == '\0' || isspace((unsigned char)text[5]))) {
315             Email e;
316             char *body = trim(text + 5);
317
318             if (parse_email(body, arrival, &e)) {
319                 arrival++;
320                 if (!heap_insert(heap, &e)) {
321                     fprintf(stderr,
322                         "Error: out of memory while storing email on line %ld.\n",
323                         line_number);
324                     return;
325                 }
326             } else {
327                 fprintf(stderr,
328                     "Warning: skipping malformed email on line %ld.\n",
329                     line_number);
330             }
331         } else if (strcmp(text, "NEXT") == 0) {
332             do_next(heap);
333         } else if (strcmp(text, "READ") == 0) {
334             do_read(heap);
335         } else if (strcmp(text, "COUNT") == 0) {
336             do_count(heap);
337         } else {
338             fprintf(stderr,
339                 "Warning: unrecognized line %ld: %s\n", line_number, text);
340         }
341     }
342 }
343
344 int main(int argc, char *argv[])
345 {
346     MaxHeap heap;
347     FILE *in = stdin;
348
349     if (argc > 2) {
350         fprintf(stderr, "Usage: %s [input-file]\n", argv[0]);
351         return 1;
352     }

```

```

353
354     if (argc == 2) {
355         in = fopen(argv[1], "r");
356         if (in == NULL) {
357             fprintf(stderr, "Error: could not open '%s'.\n", argv[1]);
358             return 1;
359         }
360     }
361
362     heap_init(&heap);
363     process_stream(in, &heap);
364     heap_free(&heap);
365
366     if (in != stdin)
367         fclose(in);
368
369     return 0;
370 }

```

Comparison of the Two AI Codes

1. Correctness

Claude: Correct. It uses a MaxHeap to prioritize emails by sender category and newest date. It also handles NEXT, READ, and COUNT.

Gemini: Correct. It uses the same MaxHeap approach and correctly handles the required commands.

Both work correctly, but Claude has better input and memory checks.

2. Execution Time

Claude: Insert and READ take $O(\log n)$, while NEXT takes $O(1)$.

Gemini: Has the same time complexity.

Both run in the same time.

3. Space Complexity

Claude: Uses $O(n)$ space to store the emails.

Gemini: Also uses $O(n)$ space.

Both have the same space complexity.

4. Maintainability

Claude: Better. It has more organized functions and safer input handling.

Gemini: Simpler, but has less error checking.

Claude is better for maintainability.

Because Claude is more maintainable and organized while having the same overall time and space complexity, I will use Claude for my assignment.

What I Changed

1. Correctness

- The original code prints “No emails to read.” for an empty READ, even though READ does not need to display anything. I changed this so an empty READ simply does nothing.
- I kept the validation for email categories and dates to prevent invalid email data from entering the heap.

2. Execution Time

- Remains $O(\log n)$ for inserting and reading emails, and $O(1)$ for NEXT.

3. Space Complexity

- Remains $O(n)$, since the program still stores all emails in the MaxHeap.

4. Maintainability

- Simplified some of the code and comments to make the program easier to understand and modify.
- Kept the heap operations separated into functions so they are easier to test and maintain.